A NOR8T SRAM Digital Compute-in-memory Macro for Sparse and Scalable edge-AI Processing

Pratham Sharma*, Mukul Lokhande*, Akash Sankhe*, Kwok Tai Chui†, Brij Bhooshan Gupta‡, and Santosh Kumar Vishvakarma* (Senior Member, IEEE)

*Department of Electrical Engineering, Indian Institute of Technology Indore, Indore, India
†Department of Electronic Engineering and Computer Science, Hong Kong Metropolitan University, Hong Kong
‡Department of Computer Science and Information Engineering, Asia University, Taiwan

2025 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS) | 979-8-3315-8907-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/APCCAS67402.2025.11377073

***Abstract*—In this work, we propose a 16Kb digital compute-in-memory macro with a novel NOR8T SRAM bitcell for efficient multi-bit MAC operations in edge-AI applications. The proposed NOR8T performs in-memory multiplication without wordline activation, enhancing energy efficiency and robustness to simultaneous row access, with improved transistor density over prior 10T/12T NOR designs. A compact Resource-Efficient Interleaved Adder Tree (REIT) employing IFA-8T and IFA-10T achieves 68% transistor density and 57% power savings over a conventional CMOS 28T full adder. The macro supports 1-16 bit precision for weights and inputs via bit-serial streaming and allows concurrent MAC and weight updates. Fully digital, it avoids analog-CIM limitations and delivers 1.63 TOPS throughput with 98.5% QoR accuracy on LeNet-5 at 2.3× lower power under 40% weight sparsity. The proposed design advances a scalable, energy-efficient DCIM architecture tailored for real-time edge intelligence.**

***Index Terms*—Compute-in-memory (CIM), Edge-AI accelerators, reconfigurable precision, multiply-and-accumulate, DNN (deep neural networks).**

I. Introduction

The swift advancement of machine learning algorithms is accelerating the evolution of AI systems, frequently outpacing human capabilities in fields like object detection, classification, reinforcement learning, and natural language processing. While these applications generally rely on cloud computing for heavy computations on large datasets, the growing need for real-time interactions and heightened privacy concerns are fostering a shift towards edge deployments with constrained resources. This transition has further boosted the extensive data collection enabled by the widespread use of smart sensors in augmented and virtual reality technologies, autonomous vehicles, and robotics, all aimed at enhancing data-driven performance between users and devices. Nevertheless, the substantial increase in data volume and computational needs highlighted by models like GPT-4, which has over 1,800 billion parameters and demands more than 750 billion MAC operations, has revealed the inherent limitations of conventional von-Neumann architectures. [1]–[4]. The memory-compute bottleneck has surfaced as a major hindrance to efficient and scalable AI processing.

Edge-AI devices demand ultra-low latency, event-driven responsiveness, high energy efficiency, and optimized compute density to support diverse AI workloads. Previous RTL-designed, precision-configurable neural network (NN) architectures have explored runtime-reconfigurable and layer-reuse accelerators for efficient matrix-vector multiplication (MVM) and multiply-accumulate (MAC) operations, core to Convolutional and fully connected layers in deep learning and transformer models [5], [6]. For instance, VGG-16 necessitates 15.5B MAC computations to infer a single 224×224 RGB image. Prior works have demonstrated that 4-bit quantization combined with 40% pruning maintains acceptable inference accuracy, while significantly decreasing computational and memory overhead, thus enhancing application performance [7]–[9]. CIM architectures bring compute within the memory array, thereby eliminating the communication bottleneck further.

SRAM has been the cache of choice in edge-AI SoCs because of its rapid access speed, excellent durability, and lack of refresh necessity. As a result, SRAM-based Compute-in-Memory (CIM) has emerged as a popular option, merging computation with memory arrays, which decreases energy expenses related to data transfer while preserving cache capabilities. [10]. Predominantly analog CIM (ACIM) demonstrated superior energy efficiency and throughput; however it incurs significant area and power penalties associated with analog and mixed-signal (AMS) data converter blocks. Furthermore, the performance is degraded by process variations, non-linearities, and charge leakage effects during matrix-vector multiplication (MVM), leading to a substantial drop in application accuracy [11]. In contrast, digital CIM (DCIM) bettered the implementation by removing data conversion circuitry and enhanced voltage scalability. The primary challenges associated with DCIM are: computations susceptible to bit-flips caused by simultaneous activation of multiple word lines connected to shared SRAM columns, and larger 10T/12T bitcell, leading to increased silicon footprint or reduced transistor density (#T/mm^2) and overall compute array compactness [12]. Furthermore, the adder tree structures used for precise digital

This work was supported by the Special Manpower Development Program for Chip to Start-Up (SMDP-C2S), Ministry of Electronics and Information Technology (MeitY), Government of India, under Grant EE-9/2/21 - R&D-E. (Corresponding author: Santosh Kumar Vishvakarma.)

This work was also supported by Hong Kong Metropolitan University, under Grant RD/2023/2.3 and National Science and Technology Council (NSTC), Taiwan, under Grant NSTC 114-2221-E-468-015.

979-8-3315-8907-3/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:01:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Performance comparison for different SoTA Bit-cells, adapted at TSMC-65nm, VDD 1.2 V

Bitcell Parameters	6T	8T	5T [14]	6T+4T [15]	1R1W 12T [16]	MUL8T [8]	XNOR-10T [17]	Proposed NOR8T
Area (μm^2)	1.75	2.35	1.55	3.67	4.2	2.52	3.78	2.43
Power (μW)	20.1	23.4	17.1	27.4	39.8	34	37.3	30.92
Operation	Storage			Storage, CIM-NOR		Storage, CIM-AND	Storage, XAC/MAC	Storage, CIM-NOR
Layout Density	High	High	Irregular	Low	Low	Medium	Medium	High
Read Delay (ps)	106.2	102.7	104.6	127.5	158	108.2	134	121.3
Write Delay (ps)	169.6	164.3	157.2	216.7	282	173.2	237.6	173.9

accumulation account for a substantial portion of power and area consumption, often up-to 65% of total power and 45% of the macro area [13]. Thus, there is substantial scope for algorithm-architecture co-design to effectively address the aforementioned challenges.

In order to tackle the previously mentioned issues, this research presents a 16kb SRAM-integrated DCIM macro, utilising a NOR8T bitcell in 65nm CMOS technology, designed specifically for effective MAC operations in edge-AI systems. The key innovations outlined in this paper consist of:

- **In-memory NOR8T bit-cell:** We introduce an NOR8T SRAM bit-cell, with 5 transistors for low-power weight storage and 3 for in-memory multiplication between the input and the stored weight. Unlike prior approaches, proposed NOR8T avoids the negation of the entire input, thereby further reducing resource overhead.
- **Resource-Efficient Interleaved Adder Tree (REIT):** A compact staggered adder tree architecture leveraging IFA-8T and IFA-10T cells is developed to mitigate the area and power overhead associated with DCIM adder trees.
- **Sparse and Scalable edge DCIM macro:** A DCIM-compatible 16Kb SRAM macro is proposed with a focus on resource efficiency, further enhanced by pre-compute sparse reduction in neural networks, leading to 2× improved energy efficiency in edge AI processing.

II. Research Gap and Our Solution

A. Background

A conventional DCIM macro [8] consists of an SRAM memory array, an address decoder, an AXI-enabled controller with read/write (R/W) peripheral circuitry, an adder tree, and an accumulator. The SRAM array is designed using a 6T latch, which stores both the data and its complement.The R/W control circuitry allows data access through the bitline (BL) and its complement, bitline bar (BLB), which are activated by wordline (WL) signals generated by the address decoder. The dual-function DCIM architecture employs the SRAM array for two main purposes: (1) storing filter weights and (2) performing in-memory MAC operations using input feature maps. This approach minimises data transfer between the storage array and external computation units, resulting in substantial reductions in energy consumption and latency, although it does lead to a greater area requirement. The adder tree is positioned adjacent to the SRAM array to combine the outputs from each row, enabled by a single-cycle accumulation.

B. Opportunities

The last decade has seen significant advancements in DCIM accelerators at different levels: algorithm-architecture co-design, macro architecture, and circuit-level enhancements, with efforts aimed at computational energy, memory bandwidth, signal-to-noise ratio (SNR), and throughput. DCIM macros eliminate the need for costly data conversion circuitry and support flexible multi-precision reconfigurable computing, unlike ACIM macros; however, this comes at the expense of additional hardware resources, including shifters, adders, and accumulators. Prior work [7] addressed it with time-shared hardware to enhance the CIM efficiency, albeit at the cost of throughput limitations. Therefore, the primary opportunities for optimisation lie within the compute-enabled bitcell and FA circuit design.

C. Prior works: Bit-cell

Prior work explored an analog XNOR-10T bitcell to perform configurable XAC / MAC computation in image classification [17]; however, the XNOR approach is inherently limited to binarized data representations [18]. Furthermore, recent works advanced with digital AND-8T bitcell designs with reduced transistor counts and fixed-point arithmetic [7], [8], [19]; These advancements lead to the practical adoption of 8T bitcell in commercial processors, such as AMD's Zen 4c [20]. However, the AND-based approach lacks native support for signed multiplication, constraining its effectiveness for complex DNNs, despite the improvements.

NOR-based in-memory logic is a preferred emerging approach due to its universal property, compatibility with pre-charged or pulsed compute schemes, and native support for signed computations, eliminating the need for normalization or scaling to match full-precision accuracy. However, SoTA

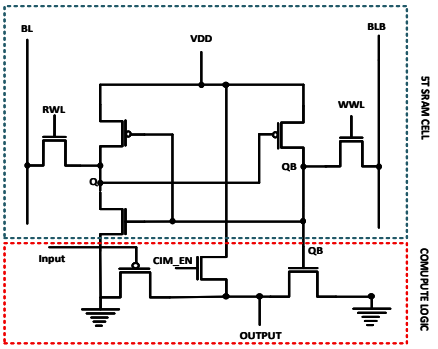


Fig. 1: The proposed NOR-8T bitcell for multiplication.

979-8-3315-8907-3/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:01:11 UTC from IEEE Xplore. Restrictions apply.

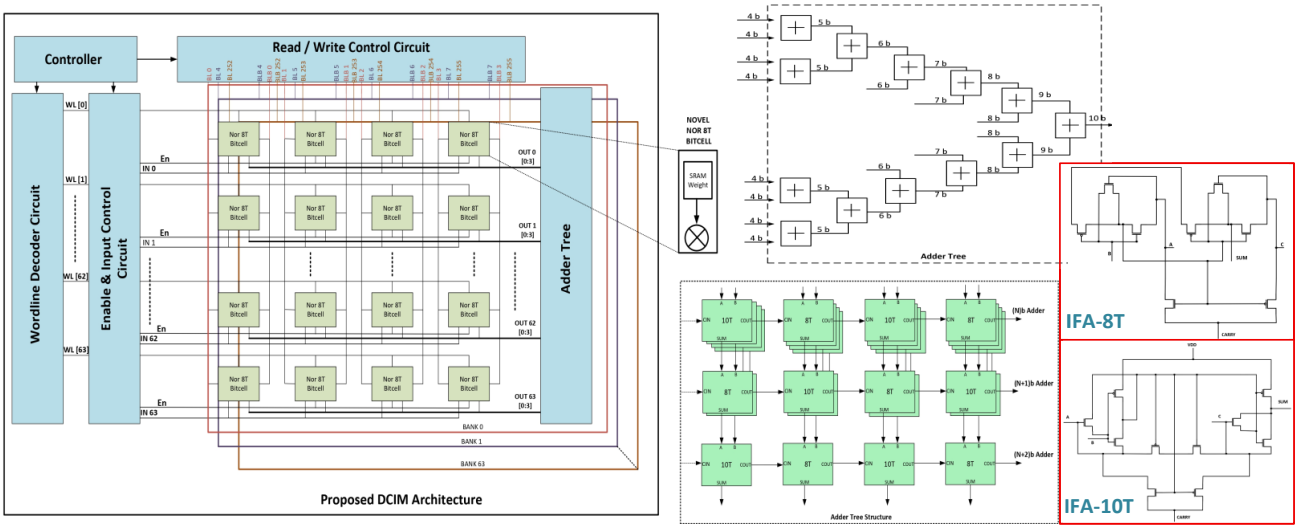


Fig. 2: (a) Proposed resource-efficient DCIM macro, (b) Proposed REIT architecture

NOR bit-cells, such as 6T+4T [15] and 1R1W-12T [16], incur significant transistor overhead, underscoring the need for optimisation in bit-cell designs. Our proposed NOR-8T (Fig. 1) addressed it with a resource-efficient design (Table I), well-suited for application requirements and practical applicability.

D. Prior works: Adder-Tree architectures

Conventional CMOS adder tree structures typically use a ripple-carry adder (RCA) approach with 28T FA, which was further power-optimised with power-gated (PG) 26T FA via minor structural modifications [7], [13], [16]. Other approaches have enhanced adder tree designs through 1D interleaving(combined 10T and 28T FAs) [21], 2D interleaving using 14T [15] or 7T [8] FAs alongside 28T FAs, the use of compressors with 12T and 16T FAs, or by replacing lower bit adders with 16T approximate adders [22] or OR logic [23] to enhance resource efficiency. The proposed solution adopts a 2D array-interleaved structure with IFA-8T and IFA-10T, providing improved power, performance, and area (PPA) efficiency without compromising the adder tree output significantly.

III. Proposed Sparse & Scalable DCIM Macro

The proposed 16Kb DCIM macro is displayed in Fig.2. The array comprises 64 banks, each with 64 rows and 4 columns (4-bit width), and is configured to store binary filter weights, similar to prior work [8]. Row-wise compute-in-memory operations are enabled at runtime via CIM_EN control signal and bank-level selection multiplexers, while input activations are streamed in a bit-serial manner (MSB to LSB), generating partial products in each clock cycle. The design supports variable-bit precision, scalable in mod-4 increments by banking, while the input bit-width requires N cycles for N-bit inputs, with a kernel (partial products) computed with a bank and accumulation via 10-bit adder tree, over N+1 clock cycles. This marks the completion of one In-memory MAC.

A. In-memory NOR8T bit-cell for DCIM computation

The proposed NOR-8T bitcell, illustrated in Fig.1, includes stored weight (Q node), input (IN), and multiplication output (MUL) for in-memory MAC operations. Built on a 5T ultra-low-power SRAM latch [14], three additional transistors enable NOR-based multiplication without interfering with word-line (WL) or bitline (BL/BLB) activity and controllability via CIM_EN signal. During storage, BL and BLB are pre-charged to VDD; in CIM mode, the MUL output is generated without activating WL, unlike prior work [4] and avoiding bit flips from simultaneous row activation to generate accurate multi-bit partial products with improved robustness. NOR-8T also allows concurrent weight updates and compute-skipping when IN = 0, enhancing both energy efficiency and throughput. This operation was validated across 100K Monte Carlo simulations, with a mean delay of 15ps, 34% better than [15].

B. Resource-Efficient Adder Tree

Prior adder tree architectures consumed 65% of total power consumption and 45% of area, marking the primary motivation

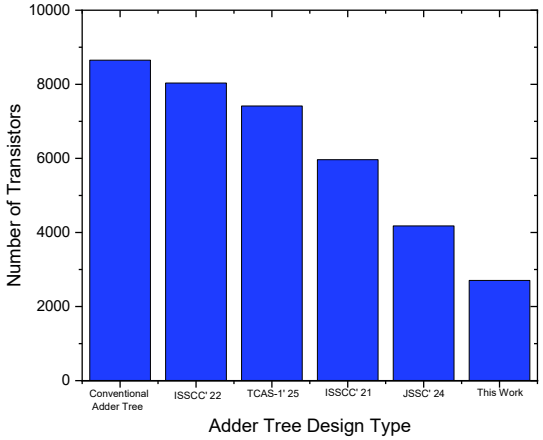


Fig. 3: Transistor Density Comparison across various adder tree architectures

979-8-3315-8907-3/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:01:11 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Performance comparison with SoTA ACIM and DCIM macros, Scaled* to macro size 16Kb

Parameters	JSSC'19 [25]	ISSCC'21 [15]	JETCAS'22 [11]	TNano'23 [17]	TCAS-I'23 [26]	TCAS-I'24 [13]	ISQED'25 [8]	DATE'25 [19]	Proposed
Tech. (nm)	65	22	65	65	65	55	65	65	65
MAC-operation	AMS	Digital	Analog	Digital	Digital	Digital	Digital	Digital	Digital
Cell-Type	10T	6T + 4T	AND8T	10T	7T	8T	MUL8T	6T + 2T	NOR8T
Supply Voltage VDD (V)	0.8-1	0.72	1	1.2	0.8-1.1	1.2	1.2	0.6-1.2	0.8-1.2
Array Size (Kb)	16	64	16	16	80	64	16	4	16
Op. Freq. (MHz)	5	100	100	25	200	200	250	40	66
Bit-cell Area (μm^2)	10.53	0.37	2.78	4.5	2.83	4.21	4.1	2.25	2.43
Input Precision (b)	6	1-8	4	4	1-16	4/8/12/16	1-8	4/8	1-16
Weight Precision (b)	1	4/8/12/16	4	1-4	1-16	4/8/12/16	4/8	4/8/12/16	1-16
Model	LeNet-5	NA	VGG-8	CNN-Type	Inception-v4	ResNet-10	LeNet-5	NA	LeNet-5
Accuracy	98.3	NA	96.05	98.67	95.3	93.7	99.1	98.5	99.14
Throughput (TOPS)	0.64	0.52*	0.41	0.82	0.41*	0.43*	2.2	2.52*	1.63
Energy Efficiency (TOPS/W)	50.6	23	180	273	63	67	480	404	321

of this work for resource optimisation. Our work introduced REIT with IFA-8T and IFA-10T, thereby improving the transistor density (Fig. 3) and CMOS-area compared to traditional 28T RCA-adder tree by 68% and power-consumption by 57% respectively. The detailed comparison with SoTA works is briefed in Table III. The proposed PG-CMOS staggered combination of IFA-8T and IFA-10T effectively mitigates threshold voltage drop and ensures stable output without exhibiting bit-flip errors in the output. The key conclusion to draw is that the static power and noise-margin remain comparable to prior works [8], despite the introduction of pass-gate logic. The introduction of REIT in DCIM macro improves the compute delay, power consumption, and consequently energy efficiency compared to SoTA works [7], [8], [13], [24].

IV. Performance Analysis & Discussion

The resource efficiency of the macro primarily stems from a reduced-transistor adder tree, power-optimised MAC operations (within the CIM array and adder tree), and skipped sparse computations. The data-flow follows a row-wise, weight-stationary scheme programmed via the controller [8], with weight precision scaled in $\lceil \frac{N}{4} \rceil$ increments across multiple memory banks, while input activations (1-16 bits) are fed bit-serially from LSB side, generating partial products each cycle. Precision is dynamically adjustable, requiring N cycles for N-bit inputs. The 64 CIM banks operate in parallel as 64 MAC units, and the partial products are accumulated using a 10-bit single-cycle adder tree for high-throughput inference.

TABLE III: Comparison of different adder tree architectures, via [27], adapted at TSMC-65nm, VDD 1.2 V

Adder Tree Structure	Avg_Error	RMSE	Power(μW)	Delay (ns)
Conv-28T	0	0	892	3.56
Proposed	0	0	369	1.98
2D-Inter-(14T+28T) [15]	0	0	2864	0.975
DACA [28]	3.96	3.75	229	3.1
DIMCA (12T+16T) [24]	4.03	6.76	251.4	4.97
SESA [29]	5.47	4.67	386.25	2.29
PG-26T [7]	0	0	586	2.5
2D-Inter-(7T+28T) [8]	0.94	2.86	511	4.4
1D-Inter-(10T+28T) [21]	1.34	3.1	673.89	5.58
Hybrid (28T + 16T) [22]	4.5	8.8	708	9.29
LOA (28T +OR) [23]	1.38	4	488	4.7
SOAFA [30]	2.92	3.83	315.74	3.67
MxA [31]	3.3	4.18	384	3.89
MLAFA [32]	1.93	2.76	330	5.3

A. DNN performance Evaluation

The proposed resource-efficient macro is designed to integrate AI functionality into Smart City applications, including surveillance, traffic monitoring, and vehicle theft detection. The primary objective is to enable real-time, energy-efficient classification of vehicles, pedestrians, and anomalous objects at traffic intersections. Due to space constraints, we only present results for the LeNet-5 model on the MNIST dataset, demonstrating inference accuracy comparable to prior works [8], [25]. The functionally equivalent model was implemented using QKeras, the FxP-math library, and Python 3.0 on the Google Colab platform. Post-training, weights were exported in CSV format and mapped to the macro for inference at 4A4W precision. Additionally, a custom Yolo-like model with 45-class support vector classifiers (SVCs) was evaluated, achieving up to 98.5% quality-of-results (QoR) at FP32-equivalent accuracy with 2.3$\times$ lower power consumption, enabled by 40% weight sparsity. The hardware performance emulation incorporates key design parameters such as DPIM array dimensions, activation/weight precision, bank utilisation per layer, and reconfigurable hardware multiplexing (26.5μW). The architecture integrates an off-chip bit-serialised ReLU activation function and control engine, similar to prior work [8]. The off-chip compute stack includes a combination accumulator (178μW), sub-sampling AAD pooling (121μW), ReLU activation (114μW), and normalisation module (135μW), all designed to support sparsity-aware layer reuse while preserving kernel-level modularity and macro-level scalability [5].

B. Hardware Metrics Evaluation & Future work

The impact of macro size, precision, and throughput for the proposed design is quantitatively compared against SoTA designs in Table II. The proposed 16Kb macro increases throughput by minimising compute delays, which leads to quicker and more efficient execution. Its energy efficiency (TOPS/W) is markedly improved thanks to the REIT and reduced macro usage in sparse tasks. These improvements culminate in a streamlined, energy-saving, and cost-effective design with a more compact silicon footprint. Next steps involve ASIC prototyping, compute density analysis, and the impact on silicon costs, and confirm real-world efficiency, for On-Device Learning (ODL) applications for user privacy and enhanced experience.

979-8-3315-8907-3/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:01:11 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. Kim and H.-J. Yoo, "An Overview of Computing-in-Memory Circuits with NVMs," *IEEE Trans. on Circuits and Systems II*, vol. 71, no. 3, pp. 1626–1631, 2024.

[2] Z. Sun, S. Kvatinsky, X. Si, A. Mehonic, Y. Cai, and R. Huang, "A full spectrum of computing-in-memory technologies," *Nature Electronics*, vol. 6, no. 11, pp. 823–835, 2023.

[3] Z. Wang, R. Yu, Z. Jia, Z. He, T. Yang, B. Gao, Y. Li, Z. Hu, Z. Hao, Y. Liu, *et al.*, "A dual-domain compute-in-memory system for general neural network inference," *Nature Electronics*, pp. 1–12, 2025.

[4] O. Krestinskaya, L. Zhang, and K. N. Salama, "Towards Efficient In-Memory Computing Hardware for Quantized Neural Networks: State-of-the-Art, Open Challenges and Perspectives," *IEEE Transactions on Nanotechnology*, vol. 22, pp. 377–386, 2023.

[5] G. Raut, S. Karkun, and S. K. Vishvakarma, "An Empirical Approach to Enhance Performance for Scalable CORDIC-Based DNNs," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, June 2023.

[6] O. Kokane, M. Lokhande, G. Raut, A. Teman, and S. K. Vishvakarma, "LPRE: Logarithmic Posit-enabled Reconfigurable edge-AI Engine," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2025.

[7] V. Sharma, X. Zhang, N. S. Dhakad, and T. T.-H. Kim, "FlexDCIM: A 400 MHz 249.1 TOPS/W 64 Kb Flexible Digital Compute-in-Memory SRAM Macro for CNN Acceleration," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1–12, 2025.

[8] A. Sankhe, M. Lokhande, R. Sharma, and S. K. Vishvakarma, "Area-Optimized 2D Interleaved Adder Tree Design for Sparse DCIM Edge Processing," in *26th International Symposium on Quality Electronic Design (ISQED)*, vol. 26, pp. 1–6, Dec. 2025.

[9] M. Lokhande, G. Raut, and S. K. Vishvakarma, "Flex-PE: Flexible and SIMD Multiprecision Processing Element for AI Workloads," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, no. 6, pp. 1610–1623, 2025.

[10] H. Diao *et al.*, "A Multiply-Less Approximate SRAM Compute-In-Memory Macro for Neural-Network Inference," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 2, pp. 695–706, 2025.

[11] V. Sharma, J.-E. Kim, H. Kim, L. Lu, and T. T.-H. Kim, "A reconfigurable 16Kb AND8T SRAM macro with improved linearity for multi-bit Compute-in-Memory of Artificial Intelligence edge devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 12, no. 2, pp. 522–535, 2022.

[12] N. S. Dhakad, E. Chittora, G. Raut, V. Sharma, and S. K. Vishvakarma, "In-Memory Computing with 6T SRAM for Multi-operator Logic Design," *Circuits Syst. Signal Process.*, vol. 43, p. 646–660, Aug. 2023.

[13] W. Yi, K. Mo, W. Wang, Y. Zhou, Y. Zeng, Z. Yuan, B. Cheng, and B. Pan, "RDCIM: RISC-V supported full-digital computing-in-memory processor with high energy efficiency and low area overhead," *IEEE Trans. on Circuits and Systems I*, vol. 71, no. 4, pp. 1719–1732, 2024.

[14] A. Teman, A. Mordakhay, J. Mezhibovsky, and A. Fish, "A 40-nm Sub-Threshold 5T SRAM Bit Cell With Improved Read and Write Stability," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 59, no. 12, pp. 873–877, 2012.

[15] Y.-D. Chih, P.-H. Lee, H. Fujiwara, *et al.*, "An 89TOPS/W and 16.3TOPS/mm^2 All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications," in *IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, pp. 252–254, 2021.

[16] H. Fujiwara, H. Mori, W.-C. Zhao, *et al.*, "A 5-nm 254-TOPS/W 221-TOPS/mm^2 Fully-Digital Computing-in-Memory Macro Supporting Wide-Range Dynamic-Voltage-Frequency Scaling and Simultaneous MAC and Write Operations," in *IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, pp. 1–3, 2022.

[17] P. K. Saragada, S. Manna, A. Singh, and B. P. Das, "A configurable 10T SRAM-based IMC accelerator with scaled-voltage-based pulse count modulation for MAC and high-throughput XAC," *IEEE Transactions on Nanotechnology*, vol. 22, pp. 222–227, 2023.

[18] N. S. Dhakad, E. Chittora, V. Sharma, and S. K. Vishvakarma, "R-inmac: 10T SRAM based reconfigurable and efficient in-memory advance computation for edge devices," *Analog Integr. Circuits Signal Process.*, vol. 116, p. 161–184, Sept. 2023.

[19] P. Tyagi and S. Mittal, "A 101 TOPS/W and 1.73 TOPS/mm^2 6T SRAM-Based Digital Compute-in-Memory Macro Featuring a Novel 2T Multiplier," in *Design, Automation & Test in Europe Conference (DATE)*, pp. 1–7, 2025.

[20] T. Burd, S. Venkataraman, W. Li, *et al.*, ""Zen 4c": The AMD 5nm Area-Optimized ×86-64 Microprocessor Core," in *IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, pp. 38–40, 2024.

[21] Z. Wang, H. Luo, Z. Peng, X. Chao, and Y. He, "An 8T SRAM Based Digital Compute-In-Memory Macro For Multiply-And-Accumulate Acceleration," in *ISCAS*, pp. 1–5, 2023.

[22] O. Kokane, P. Sati, M. Lokhande, and S. K. Vishvakarma, "HOAA: Hybrid Overestimating Approximate Adder for Enhanced Performance Processing Engine," in *28th International Symposium on VLSI Design and Test (VDAT)*, pp. 1–6, 2024.

[23] A. Dalloo, A. Najafi, and A. Garcia-Ortiz, "Systematic Design of an Approximate Adder: The Optimized Lower Part Constant-OR Adder," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 8, pp. 1595–1599, 2018.

[24] C.-T. Lin, D. Wang, B. Zhang, *et al.*, "DIMCA: An Area-Efficient Digital In-Memory Computing Macro Featuring Approximate Arithmetic Hardware in 28 nm," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, 2024.

[25] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, 2019.

[26] H. Kim, J. Mu, C. Yu, T. T.-H. Kim, and B. Kim, "A 1-16b Reconfigurable 80Kb 7T SRAM-Based Digital Near-Memory Computing Macro for Processing Neural Networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 4, pp. 1580–1590, 2023.

[27] Y. Arbeletche, G. Paim, B. Abreu, S. Almeida, E. Costa, P. Flores, and S. Bampi, "MAxPy: A Framework for Bridging Approximate Computing Circuits to Its Applications," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 11, pp. 4748–4752, 2024.

[28] X. Fan, T. Zhang, H. Li, H. Liu, S. Lu, and J. Han, "DACA: Dynamic Accuracy-Configurable Adders for Energy-Efficient Multi-Precision Computing," *IEEE Transactions on Nanotechnology*, vol. 22, pp. 400–408, 2023.

[29] C. K. Jha, A. Nandi, and J. Mekie, "Single Exact Single Approximate Adders and Single Exact Dual Approximate Adders," *IEEE Trans. on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 7, pp. 907–916, 2023.

[30] C. He, Z. Wang, F. Xiang, Z. Dai, Y. He, J. Yue, and Y. Liu, "LSAC: A Low-Power Adder Tree for Digital Computing-in-Memory by Sparsity and Approximate Circuits Co-Design," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 2, pp. 852–856, 2024.

[31] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-Power Digital Signal Processing Using Approximate Adders," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 1, pp. 124–137, 2013.

[32] Z. Chu, C. Shang, T. Zhang, Y. Xia, L. Wang, and W. Liu, "Efficient Design of Majority-Logic-Based Approximate Arithmetic Circuits," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 12, pp. 1827–1839, 2022.

979-8-3315-8907-3/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:01:11 UTC from IEEE Xplore. Restrictions apply.